COP2034C Module 7 Programming Assignment

Instructions

In this assignment, we will continue building the Palmer Penguins program from Module 6. In that module, we reorganized the program using functions so that individual calculations and operations could be performed by smaller, reusable parts of the program.

In Module 7, we will begin working with text from the actual Palmer Penguins dataset. Real datasets are commonly stored in files in which each row contains several pieces of information. Before Python can analyze those values, the program often needs to separate and clean the text.

We will practice selected Chapter 7 concepts:

- using string methods,
- using split() to separate a string into individual values,
- accessing the resulting values by index,
- converting numeric strings to numeric values,
- using string formatting to display data in columns,
- and using functions to organize string-processing operations.

The Chapter 7 split() method is particularly useful for data processing because it converts one string into a list of tokens based on a separator.

We are not learning Python file processing yet. The "read_penguin_row" function needed in this assignment for retrieving rows from the CSV file is provided for you as a .py file in the assignment repository. You can copy and paste into your notebook. Your responsibility is to work with the data returned by that function.

As before, work incrementally. Make one change, run the program, verify the result, and then continue.

1. Start with Your Module 6 Solution

Open the Palmer Penguins notebook you completed for Module 6. Make a copy for Module 7.

A suggested notebook name is:

```
PalmerPenguinsM7.ipynb
```

The Python file you eventually download should have a meaningful name such as:

```
PalmerPenguinsM7.py
```

Module 7 should build on your completed Module 6 program.

2. Update the ID Header

Update the ID header at the top of the program:

```
# PalmerPenguinsM7.py
# Your Name
# Submission Date
# Process Palmer Penguin data using String operations
```

Leave a blank line after the header.

3. Retain Your Module 6 Program

Keep the following existing Module 6 components:

- constants,
- functions,
- parallel lists,
- dataset calculations,
- formatted output,
- interactive species exploration,
- main() function,
- and call to main().

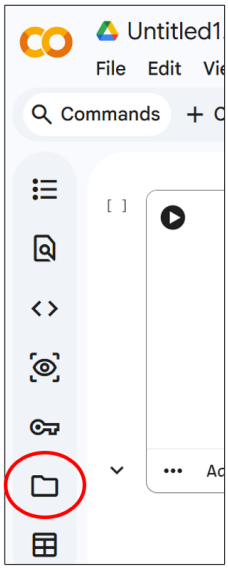
The Module 6 solution already uses functions such as calculate_percentage(), classify_species(), get_position_description(), and display_species_statistics().

We will add a small new section that works with individual records from the actual dataset.

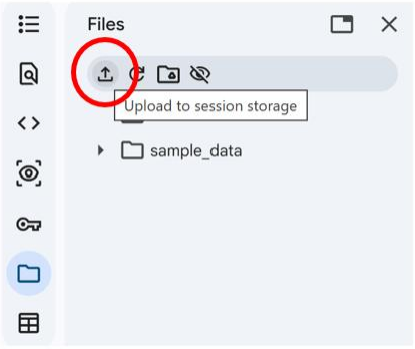
4. Upload the Palmer Penguins CSV File

Download the "PalmerPenguins.csv" file provided in the GitHub repository, then upload it to your notebook environment using the following steps:

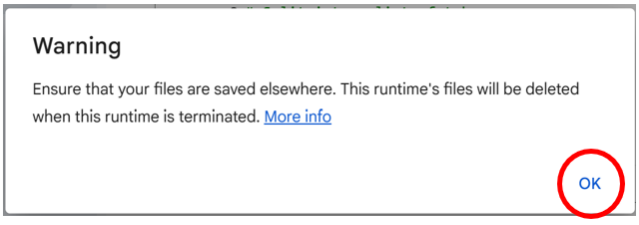
a. Click on the Files folder icon (circled in red, below):



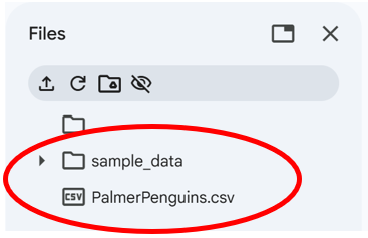
b. Once your notebook is connected to a runtime (this may take several seconds if you are not already connected), click on the arrow to upload the file to session storage:



c. You will see this warning, which explains that if your session disconnects you will have to repeat the above steps to re-upload the file. Click OK to continue.



d. Verify the file was uploaded correctly. Your Python code can now access the data file.



Each row in the file represents one penguin observation. The values on a row are separated by commas. For example, a simplified row looks like this:

```
1,Adelie,39.1,18.7,181,3750,male
```

This is one string containing several related pieces of data.

Important: You are not responsible for writing the function that reads the file; that code has been provided. File processing will be introduced later in the course. **Do not modify the provided code!**

5. Add the File-Reading Function

You can open the "read_penguin_row.py" file directly in the GitHub assignment repository and copy and paste the provided code to the code block in your notebook (use Ctrl-A/Ctrl-C to select all/copy from the provided file, then Ctrl-Shift-v to paste into the notebook). You can also download the file from the repo and use your editor of choice to copy from.

The function retrieves a row from the file using the call

```
penguin_row = read_penguin_row(row_number)
```

The function returns the requested row as a string. Again, **do not modify this function.** Its purpose is simply to give us some realistic string data to process.

6. Examine a Penguin Record

Inside main(), retrieve one penguin observation using the provided function.

```
penguin_row = read_penguin_row(1)
```

Display the original row:

```
print("Sample dataset row:")
print(penguin_row)
```

At this point, the entire row is still one string.

7. Split the Record into Individual Values

Chapter 7 introduced **split()**, which separates a string into tokens and returns those tokens in a list. Since CSV data uses commas to separate values, use:

```
penguin_data = penguin_row.split(",")
```

The resulting list will contain the individual fields from the row. For example,

```
penguin_data[0]
```

might contain the sample number, while

```
penguin_data[1]
```

might contain the species. Display the resulting list temporarily while developing your program:

```
print(penguin_data)
```

You can remove or comment out this line before submitting. This demonstrates an important data-processing operation:

CSV row string
↓
split(",")
↓
list of individual values

8. Extract Selected Penguin Values

Use indexes from penguin_data to retrieve selected fields from the observation. Create appropriately named variables for:

```
sample number
species
bill length
bill depth
body mass
sex
```

For example:

```
species = penguin_data[1]
```

Use the correct indexes for the remaining values based on the provided CSV file. This is similar to the list indexing you have already used. The difference is that the list is created by splitting a string rather than by writing the values directly in the program.

9. Convert Numeric Data

Values produced by split() are strings, even when the characters represent numbers. Convert the appropriate values before using them as numeric data. For example,

```
sample_number = int(penguin_data[0])
species = penguin_data[1]
```

and

```
bill_length = float(penguin_data[2])
```

and

```
body_mass = int(penguin_data[5])
```

Use the appropriate conversion for each numeric value. This is an important distinction when working with datasets: the text "3750" and the integer 3750 may look similar when displayed, but Python treats them as different types.

10. Create a Function to Display the Penguin Observation

Create a function named:

```
display_penguin_observation()
```

Pass this function the required parameters to display the selected penguin. Use formatted output so that the labels and values are easy to read. For example, the output might look like this:

```
Sample Penguin Observation

Sample Number: 1
Species:       Adelie
Bill length:   39.1 mm
Bill depth:    18.7 mm
Body mass:     3750 g
Sex:           male
```

Use an f-string field width to align the labels or values rather than inserting a different number of spaces into each string.

11. Process Another Observation

Retrieve and process a second row from the dataset. Use the same string-processing statements and functions to display the second observation. Do not create separate processing functions for Adelie, Chinstrap, and Gentoo penguins. The goal is to demonstrate that the same operations can process different observations from the dataset. Choose a different species from the first row.

12. Keep the Existing Species Exploration

After displaying the sample observations, keep the interactive species exploration from Module 6. The user should still be able to enter:

Chinstrap
Gentoo
Adelie
Done

and receive the existing species-level information. The CSV portion of this assignment adds individual penguin observations to our analysis. It does not replace the species-level statistics that we have been developing throughout the previous modules.

Expected Program Organization

Your completed program should have the general organization:

```
ID header

import statements

constants

provided file-reading function

function definitions
    calculate_percentage()
    classify_species()
    get_position_description()
    display_species_statistics()
    display_penguin_observation()

main()
    create species lists
    calculate dataset statistics
    display dataset statistics

    retrieve a CSV row
    split the row
    extract and convert values
    display the observation

    retrieve another CSV row
    split the row
    extract and convert values
    display the observation
```

```
    explore species using the existing while loop
    display final exploration summary

main()
```

Expected Output

The beginning of the program should continue to display the species-level information from Module 6. The existing Module 6 output includes the three species counts and percentages and retains the interactive exploration loop. The program should now also display information from two individual observations in the CSV dataset, like this:

```
Sample Penguin Observation

Species:       Adelie
Bill length:   39.1 mm
Bill depth:    18.7 mm
Body mass:     3750 g
Sex:           male

Sample Penguin Observation

Species:       Gentoo
Bill length:   ...
Bill depth:    ...
Body mass:     ...
Sex:           ...
```

The exact values depend on the rows specified in the assignment. Minor whitespace differences are acceptable as long as the output is readable and contains the required information.

Test Your Program

Before submitting, verify that:

- the original Module 6 species statistics still display correctly,
- two rows are successfully retrieved from the CSV file,
- split(",") separates each row into individual values,
- the required numeric strings are converted to numeric types,
- the observation output is aligned using formatted field widths,
- two different penguin observations can be processed using the same code,
- the existing species exploration still works,
- and Done still ends the interactive loop.

Data Science Connection

Until now, our Palmer Penguins program has worked primarily with summary information that was placed directly into the program, such as the total number of Adelie, Chinstrap, and Gentoo observations.

This module begins the transition from programming with predefined values to working with actual dataset observations. CSV (comma-separated values) files are commonly used to exchange and store tabular data. Each line represents an observation, while commas separate the individual variables associated with that observation. The Chapter 7 split() operation lets us transform one line of text into a list containing those individual values.

This illustrates a basic data-science workflow:

```
raw data →
parse values →
convert data types →
analyze →
display results
```

For example, a body mass stored in a CSV file initially enters our program as text. Before we could eventually calculate an average body mass, compare body masses, or find the largest penguin, that text must be converted into numeric data. In later modules, we will expand this process from examining a few observations to processing larger portions of the dataset.

Readability

Continue following the readability expectations from previous modules:

- meaningful variable and function names,
- consistent four-space indentation,
- reasonable line lengths,
- logical blank lines,
- and comments identifying major sections.

Useful new comments might include:

```
# retrieve sample penguin data
# split the CSV row into individual values
# convert numeric values
# display the penguin observation
```

Do not duplicate code unnecessarily. The same functions should be usable with different penguin observations.

Screenshot and Submission

Follow the same submission process used for Module 6.

Run the completed program in Google Colab. For the screenshot, show enough of one execution to demonstrate:

- the output from at least one CSV observation,
- the existing interactive species exploration,
- and the final exploration summary.

Suggested filenames:

```
PalmerPenguinsM7.py
PalmerPenguinsM7.png
```

Upload both files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.